Lecture 5 : Transformation par modèle (M2M) avec ATL

Ingénierie Dirigée par les Modèles (IDM / MDE) – version 2027

Des familles aux personnes : règles, helpers et exécution dúne transformation ATL

Pr. Aissam BERRAHOU

Département Ingénierie des systèmes embarqués et mobiles – Smart Systems Lab
ENSIAS, Université Mohammed V de Rabat

aissam.berrahou@ensias.um5.ac.ma

Feuille de route

- **Pourquoi M2M ?** : *Qu'est-ce qu'une transformation de modèle à modèle et à quoi sert-elle ?*
- **Le langage ATL** : *Comment est organisé un programme ATL (module, en-tête, OCL) ?*
- **Règles et helpers** : *Comment écrire une règle et un helper ? Comment ATL les exécute-t-il ?*
- **Cas complet : Families2Persons** : *Comment réaliser, de bout en bout, la transformation Families2Persons ?*
- **Aller plus loin et synthèse** : *Comment relier les éléments créés ? Quelles erreurs éviter ? Que retenir ?*

Objectifs de la séance

À la fin de cette séance, vous saurez :

- Expliquer ce qu'est une transformation **modèle vers modèle** (M2M) et la situer dans **MDA**.
- Décrire les éléments d'une transformation : **méta-modèles** source et cible, **modèle** source, **modèle** cible, **module** ATL.
- **Lire** et **écrire** un module ATL : en-tête, **helpers** et **règles** (`from`, filtre, `to`, liaisons `<-`).
- Expliquer comment le **moteur ATL** exécute les règles et **relie** automatiquement les éléments créés.
- Réaliser dans Eclipse la transformation **Families2Persons** du tutoriel (TP).

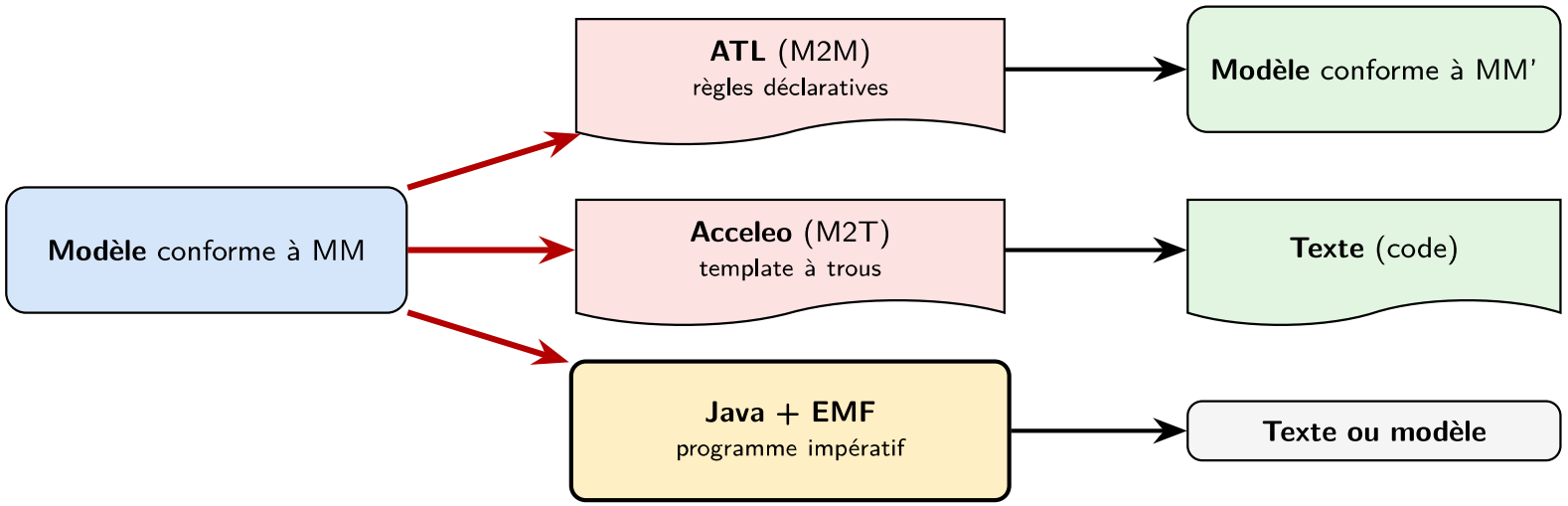
> **!** Lecture 1 (modèle, méta-modèle, Ecore), Lecture 2 (MDA : PIM, PSM, transformations) et les bases d'**OCL**.

Pourquoi M2M ?

Qu'est-ce qu'une transformation de modèle à modèle et à quoi sert-elle ?

Rappel : trois façons de transformer un modèle

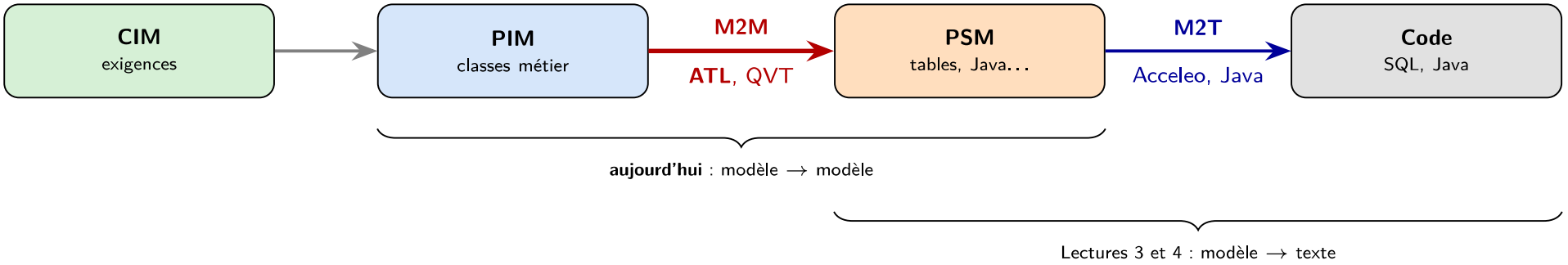
→ Lectures 3 et 4... et aujourd'hui



- **ATL** (aujourd'hui) produit un **modèle** : le résultat est conforme à un **méta-modèle**, on peut encore le transformer.
- **Acceleo** (Lecture 3) et la **programmation** (Lecture 4) servent surtout à produire du **texte** (code).

Où se situe la transformation M2M dans MDA ?

→ Entre les marches de l'escalier MDA



- Une transformation **M2M** fait passer d'un modèle à un **autre modèle** : typiquement du **PIM** au **PSM**.
- Exemple vu en Lecture 2 : un diagramme de classes (PIM) devient un **schéma relationnel** (PSM).
- La M2T vient **après** : elle écrit le code à partir du PSM.

Transformation de modèle (M2M)

Programme qui, à partir d'un **modèle source** conforme à un **méta-modèle source**, produit automatiquement un **modèle cible** conforme à un **méta-modèle cible**, en appliquant des **règles de correspondance**.

Exemples

- Familles → Personnes (le tutoriel).
- Classes UML → Tables relationnelles.
- Machine à états → Réseau de Petri.
- PIM Library → PSM Java EE.

Ce qu'on écrit

- **Pas** une boucle sur le modèle : des **règles** « tel élément source donne tel élément cible ».
- Les règles sont écrites **au niveau des méta-modèles** (M2) : elles valent pour **tous** les modèles (M1).

Le vocabulaire des transformations

→ Pour classer une transformation

Terme	Signification	Exemple
Exogène	Méta-modèles source et cible **différents**	Families → Persons ; UML → Relationnel
Endogène	**Même** méta-modèle en entrée et en sortie	Refactoring d'un diagramme de classes
Verticale	Change de **niveau d'abstraction**	PIM → PSM (on ajoute des détails techniques)
Horizontale	Reste au **même niveau** d'abstraction	Migration d'un modèle UML 1 vers UML 2
Unidirectionnelle	Ne fonctionne que dans **un sens** (cas d'ATL)	IN → OUT uniquement



ATL écrit des transformations **unidirectionnelles** : le modèle source est **lu seulement**, le modèle cible est **écrit seulement**.

Pourquoi un langage dédié ?

→ On pourrait écrire la transformation en Java... mais

Plus court

On décrit **ce qui correspond** à quoi, pas **comment** parcourir le modèle : pas de boucles, pas de chargement de fichiers.

Plus sûr

Le moteur **vérifie** les types grâce aux méta-modèles et **relie** automatiquement les éléments créés (trace).

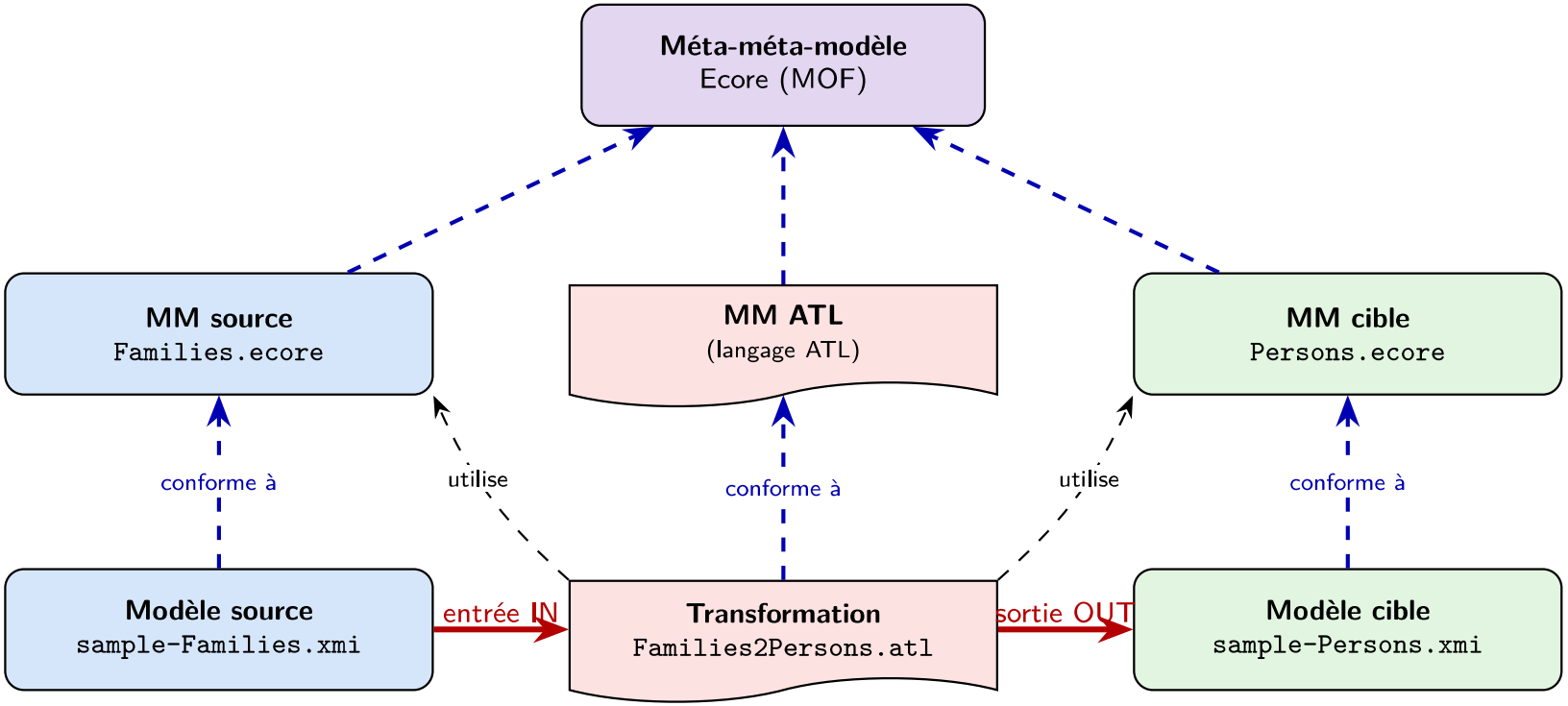
Un modèle lui-même

Une transformation ATL est **conforme au méta-modèle d'ATL** : on peut l'analyser, la transformer, la réutiliser.

! Transformer un modèle, c'est encore **modéliser** : on écrit des **règles** sur les méta-modèles, et un moteur les applique.

Le schéma général d'une transformation ATL

→ Six fichiers, trois niveaux



*La transformation est elle-même un **modèle** : tout est conforme à Ecore.*

- Le **moteur ATL** lit le modèle source (IN) et produit le modèle cible (OUT) en appliquant le module `Families2Persons.atl`.

Question flash

?

Classez ces transformations (exogène ou endogène ; verticale ou horizontale) : **(a)** Families → Persons ; **(b)** supprimer les classes vides d'un diagramme de classes ; **(c)** diagramme de classes (PIM) → schéma relationnel (PSM).

Question flash

?

Classez ces transformations (exogène ou endogène ; verticale ou horizontale) : **(a)** Families → Persons ; **(b)** supprimer les classes vides d'un diagramme de classes ; **(c)** diagramme de classes (PIM) → schéma relationnel (PSM).

Réponse

(a) exogène (deux méta-modèles), horizontale (même niveau d'abstraction). **(b)** endogène (même méta-modèle UML), horizontale. **(c)** exogène et **verticale** : on passe d'un modèle indépendant de la plate-forme à un modèle spécifique (SQL).

Le langage ATL

Comment est organisé un programme ATL (module, en-tête, OCL) ?

ATL en bref

→ ATLAS Transformation Language

- Créé par l'équipe **ATLAS** (INRIA / Université de Nantes), aujourd'hui projet **Eclipse** (MMT).
- Langage **hybride** : surtout **déclaratif** (règles), avec une partie impérative si nécessaire.
- Les expressions sont écrites en **OCL** (Lecture OCL) : navigation, collections, `if ... endif`.
- Intégré à Eclipse : éditeur avec coloration, complétion, lancement par **Run**.
- Fichiers : `.atl` (source) compilé en `.asm` (exécuté par la machine virtuelle ATL).



Eclipse → Help → Install New Software → site de mise à jour d'ATL (page **eclipse.dev/atl/downloads**), ou Eclipse Marketplace.



La transformation **Families2Persons** du tutoriel (TP3), utilisée dans tout le cours, le TD et le TP.

Le problème du tutoriel

→ Des familles. . . vers une simple liste de personnes

Source : une liste de familles

- Chaque **famille** a un **nom** (`lastName`).
- Elle comprend un **père**, une **mère**, des **fils** et des **filles** (0, 1 ou plus).
- Chaque **membre** a un **prénom** (`firstName`).

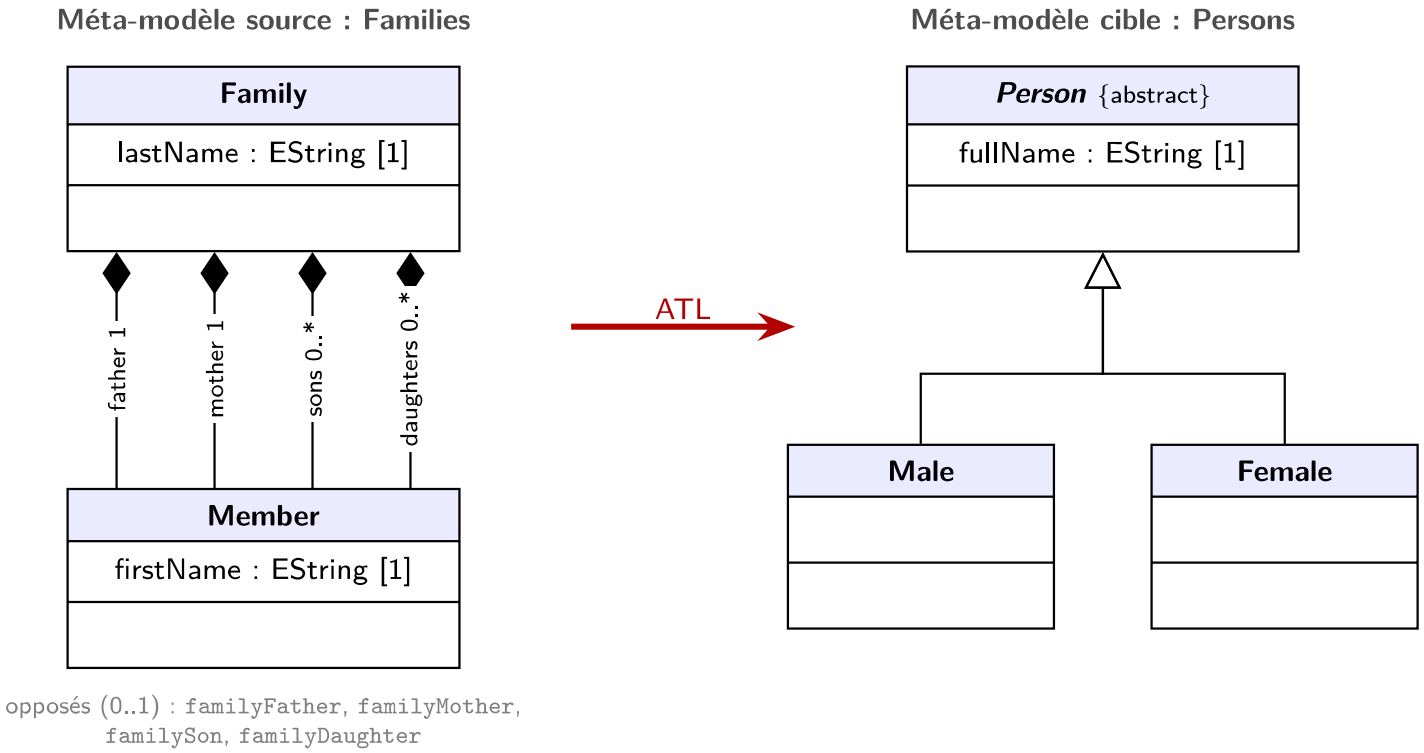
Cible : une liste de personnes

- Chaque membre devient **une personne**, sans distinguer parents et enfants.
- La personne a un **nom complet** : prénom + nom (`fullName`).
- Elle est soit **Male**, soit **Female**.

Exemple : dans la famille March, le père Jim devient l'homme « Jim March » et la fille Brenda devient la femme « Brenda March ».

Les deux méta-modèles

→ Ce que l'on a (Families) et ce que l'on veut (Persons)

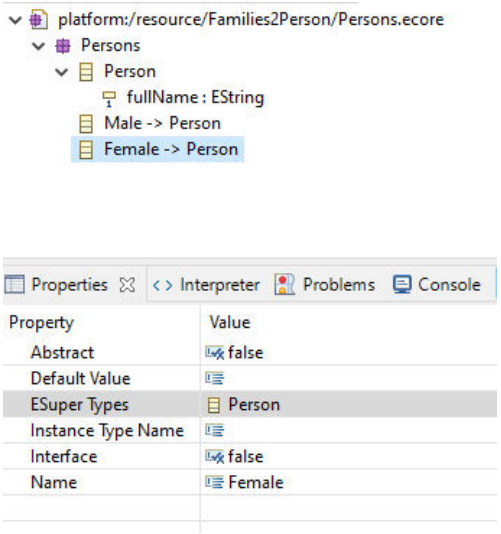


- `father`, `mother`, `sons`, `daughters` sont des **compositions** ; chacune a une référence **opposée** dans `Member` (`familyFather`...) qui indique le **rôle** du membre.
- `Person` est **abstraite** : on ne crée que des `Male` ou des `Female`.

Les méta-modèles dans Eclipse

→ Captures du tutoriel (éditeurs Ecore et OCLinEcore)

```
package Families : Families = 'http://www.eclipse.org/sirius/sample/Families'
{
    class Family
    {
        attribute lastName : String[1];
        property father#familyFather : Member[1] { composes };
        property mother#familyMother : Member[1] { composes };
        property daughters#familyDaughter : Member[*|1] { composes };
        property sons#familySon : Member[*|1] { composes };
    }
    class Member
    {
        attribute firstName : String[1];
        property familyFather#father : Family[?];
        property familyMother#mother : Family[?];
        property familySon#sons : Family[?];
        property familyDaughter#daughters : Family[?];
    }
}
```



- À gauche : `Families.ecore` affiché avec l'éditeur **OCLinEcore** (texte).
- À droite : `Persons.ecore` dans l'éditeur arborescent : `Male -> Person`, `Female -> Person` (super-type).

Structure d'un module ATL

→ Quatre parties, toujours dans cet ordre

```
-- @path Families=/Families2Persons/Families.ecore
-- @path Persons=/Families2Persons/Persons.ecore

module Families2Persons;
create OUT : Persons from IN : Families;

helper context Families!Member def: isFemale() : Boolean = ...;
helper context Families!Member def: familyName : String = ...;

rule Member2Male { from ... to ... }
rule Member2Female { from ... to ... }
```

① **Chemins** des méta-modèles (aide de l'éditeur)

② **En-tête** : modèle produit (OUT) à partir du modèle lu (IN)

③ **Helpers** : « fonctions » réutilisables

④ **Règles** : ce qui est créé dans la cible

! Le nom du **module** doit être celui du **fichier** : `module Families2Persons;` dans `Families2Persons.atl`.

L'en-tête du module

→ Qui est lu ? Qui est produit ?

```
-- @path Families=/Families2Persons/Families.ecore
-- @path Persons=/Families2Persons/Persons.ecore

module Families2Persons;                    -- nom du module = nom du fichier
create OUT : Persons from IN : Families;    -- OUT conforme à Persons, IN à Families
```

- `IN` et `OUT` sont des **noms de modèles** (on peut en choisir d'autres) ; `Families` et `Persons` sont des **noms de méta-modèles**.
- Pour désigner une classe d'un méta-modèle : `Méta-modèle!Classe`, par exemple `Families!Member`, `Persons!Female`.
- Les lignes `-- @path` (commentaires spéciaux) permettent à l'éditeur de trouver les méta-modèles : **complétion** et vérification.

Les expressions : de l'OCL

→ **Tout ce qui est calculé en ATL est une expression OCL**

Besoin	**Écriture ATL / OCL**	**Remarque**
Lire un attribut, une référence	`s.firstName`, `s.familyFather`	Navigation pointée
Tester « pas de valeur »	`s.familyMother.oclIsUndefined()`	Équivaut à « est null »
Concaténer des chaînes	`s.firstName + ' ' + s.familyName`	Chaînes entre **apostrophes**
Condition	`if c then a else b endif`	Le `else` est **obligatoire**
Logique	`not`, `and`, `or`	`not s.isFemale()`
Taille, filtre d'une collection	`f.sons->size()`, `c.attributes->select(a | a.isKey)`	Flèche `->` sur les collections
Transformer une collection	`f.sons->collect(m | m.firstName)`	Résultat : une collection
Tous les objets d'une classe	`Families!Member.allInstances()`	Dans le modèle source

Naviguer dans le modèle source

→ Valeurs sur la famille March (m1 = Jim, m4 = Brenda)

Expression	Résultat
`m1.firstName`	`'Jim'`
`m1.familyFather`	la famille `f1` (Jim est le **père** de f1)
`m1.familyMother.oclIsUndefined()`	`true` (Jim n'est pas une mère)
`m4.familyDaughter.lastName`	`'March'`
`f1.sons->size() + f1.daughters->size()`	2 (Brandon et Brenda)
`f1.sons->collect(m | m.firstName)`	`Sequence{'Brandon'}`
`Families!Family.allInstances()->size()`	2 (March et Sailor)



Pour connaître le **rôle** d'un membre, il suffit de regarder **quelle** référence opposée est renseignée.

Question flash

?

Dans l'en-tête `create OUT : Persons from IN : Families;`, que désignent `OUT`, `Persons`, `IN` et `Families` ? Et que vaut `m4.familyFather.oclIsUndefined()` pour Brenda ?

Question flash

? Dans l'en-tête `create OUT : Persons from IN : Families;`, que désignent `OUT`, `Persons`, `IN` et `Families` ? Et que vaut `m4.familyFather.oclIsUndefined()` pour Brenda ?

Réponse

`IN` et `OUT` sont les **modèles** (source et cible) ; `Families` et `Persons` sont leurs **méta-modèles**. Brenda est une **fille** : `familyFather` n'est pas renseignée, donc `oclIsUndefined()` vaut `true`.

Règles et helpers

Comment écrire une règle et un helper ? Comment ATL les exécute-t-il ?

Anatomie d'une règle (matched rule)

→ Une règle = « tel élément source donne tel élément cible »

```
rule Member2Female {
  from
    s : Families!Member (s.isFemale())
  to
    t : Persons!Female (
        fullName <- s.firstName + ' ' + s.familyName
    )
}
```

① **Nom** de la règle (unique dans le module)

② **Motif source** : pour **chaque** `Member` ...
③ ...qui vérifie le **filtre** (garde OCL)

④ **Motif cible** : on crée **un** objet `Female`

⑤ **Liaison** (binding) : `attribut_cible <-` `expression OCL sur s`

! « **Pour chaque** `Member` **qui est une femme**, **créer** une `Female` **dont** le `fullName` vaut prénom + espace + nom de famille. »

Le motif source et le filtre

→ La partie from

```
from
    s : Families!Member (not s.isFemale())
```

- `s` est une **variable** : elle désigne, tour à tour, **chaque** élément de type `Member` du modèle source.
- Entre parenthèses : le **filtre** (ou garde), une expression OCL **booléenne**.
- Sans filtre, la règle s'applique à **tous** les `Member`.

Règle d'or

Un élément source ne doit être pris que par **une seule** règle : les filtres de `Member2Male` et `Member2Female` sont **complémentaires** (`not s.isFemale()` / `s.isFemale()`).

! L'élément n'est **pas transformé** (c'est le cas de `Family` dans le tutoriel).

Le motif cible et les liaisons

→ La partie to

```
to
    t : Persons!Female (
        fullName <- s.firstName + ' ' + s.familyName
    )
```

- `t` est l'élément **créé** dans le modèle cible ; son type doit être **concret** (`Female`, pas `Person` qui est abstraite).
- Chaque **liaison** `propriété <- expression` donne une valeur à une propriété de `t`.
- L'expression est calculée sur l'élément **source** `s` ; plusieurs liaisons sont séparées par des **virgules**.
- Une règle peut créer **plusieurs** éléments cibles : `to t1 : ...(...), t2 : ...(...)`.

Les deux règles du tutoriel

→ Families2Persons.atl – partie « règles »

```
rule Member2Male {
    from
        s : Families!Member (not s.isFemale())
    to
        t : Persons!Male (
            fullName <- s.firstName + ' ' + s.familyName
        )
}

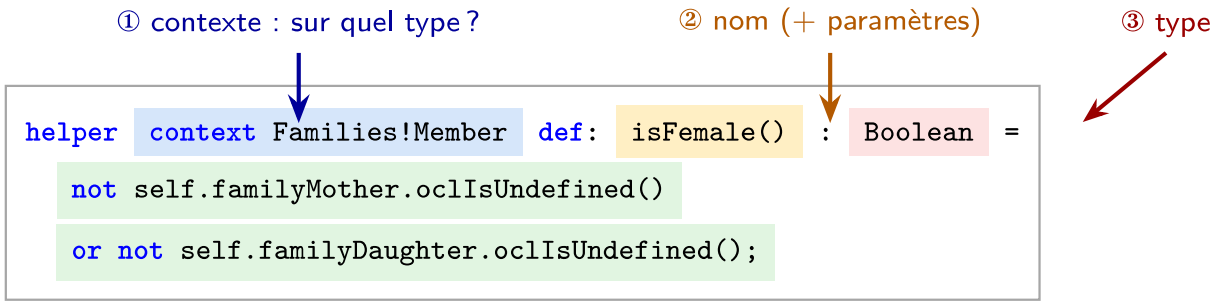
rule Member2Female {
    from
        s : Families!Member (s.isFemale())
    to
        t : Persons!Female (
            fullName <- s.firstName + ' ' + s.familyName
        )
}
```

- Deux règles, deux filtres **complémentaires** : chaque membre est pris **exactement une fois**.
- Le code des liaisons est identique : seul le **type créé** change.
- `s.isFemale()` et `s.familyName` sont des **helpers** : ils rendent les règles courtes et lisibles.

! Mettre la logique compliquée dans des **helpers**, garder les **règles** simples.

Les helpers

→ Des « fonctions » OCL attachées à un type du méta-modèle



- Le **contexte** indique sur quel type on appelle le helper ; dans le corps, `self` désigne cet objet.
- Un helper ne **modifie** rien : il **calcule** une valeur (comme une requête).
- Appel : `s.isFemale()`, comme une opération du méta-modèle.

Helper « opération » ou helper « attribut » ?

→ Avec ou sans parenthèses

	Helper opération	**Helper attribut**
Déclaration	`def: isFemale() : Boolean = ...`	`def: familyName : String = ...`
Paramètres	Possibles : `def: nom(p : String) : ...`	Aucun
Appel	`s.isFemale()`	`s.familyName`
Calcul	À **chaque** appel	Une **seule** fois par objet (mis en cache)
Sans contexte	`helper def: ...` s'appelle avec `thisModule.nom()`	`thisModule.nom`



Les deux font la même chose ici ; le helper **attribut** est un peu plus efficace quand on l'utilise souvent.

Les helpers du tutoriel

→ isFemale() et familyName

```
helper context Families!Member def: isFemale() : Boolean =
    if not self.familyMother.oclIsUndefined() then
        true
    else
        if not self.familyDaughter.oclIsUndefined() then
            true
        else
            false
        endif
    endif;
```

Version plus courte (même résultat)

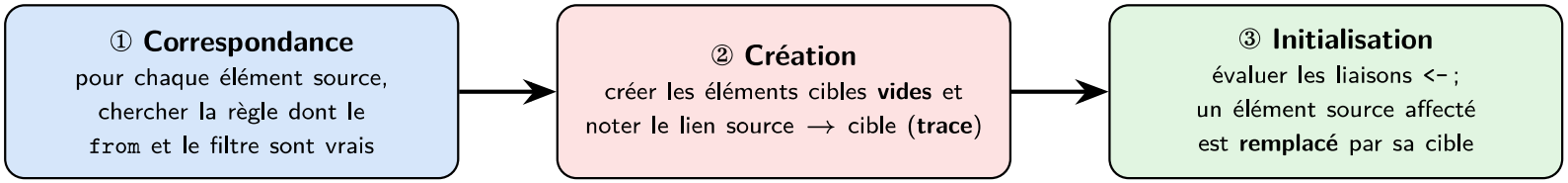
```
helper context Families!Member def: isFemale() : Boolean =
    not self.familyMother.oclIsUndefined()
    or not self.familyDaughter.oclIsUndefined();
```

```
helper context Families!Member def: familyName : String =
    if not self.familyFather.oclIsUndefined() then
        self.familyFather.lastName
    else
        if not self.familyMother.oclIsUndefined() then
            self.familyMother.lastName
        else
            if not self.familySon.oclIsUndefined() then
                self.familySon.lastName
            else
                self.familyDaughter.lastName
            endif
        endif
    endif;
```

`familyName` cherche la famille dans la référence **renseignée** (père, mère, fils ou fille) et renvoie son `lastName`.

Comment le moteur exécute une transformation

→ Trois phases, sans ordre à écrire

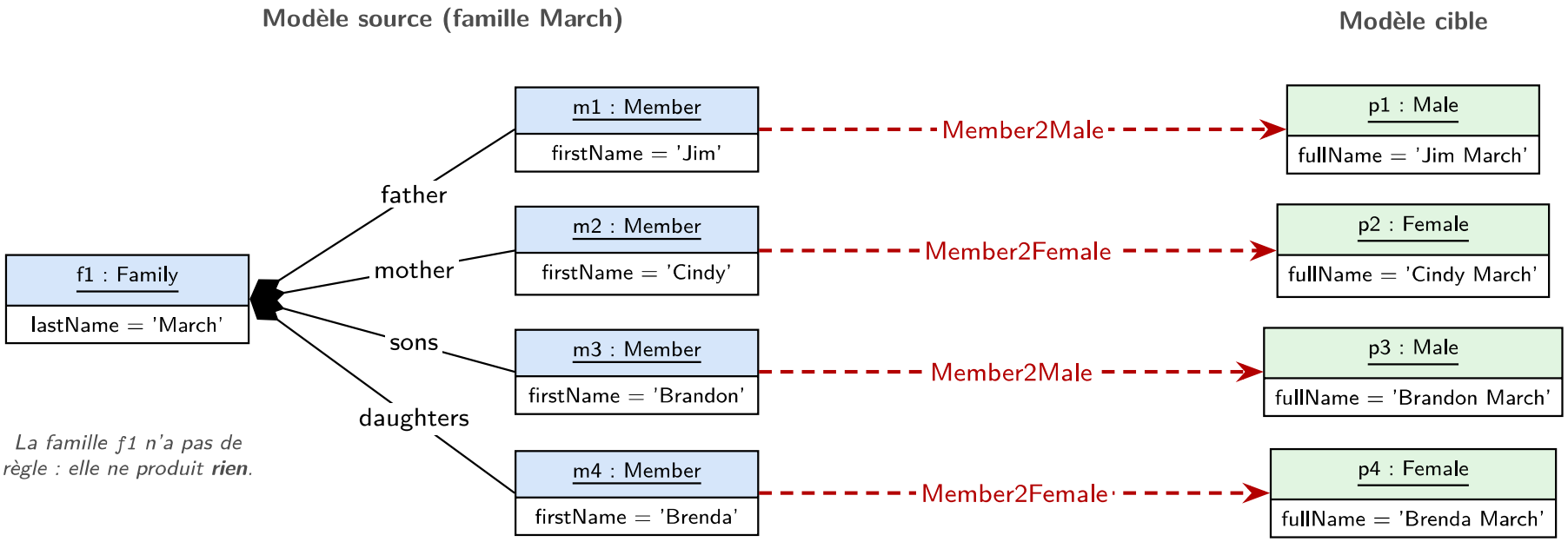


*Le programmeur ne dit **pas** dans quel ordre appliquer les règles : le moteur ATL s'en charge (langage **déclaratif**).*

- Phase ① : on **regarde** chaque élément source (`Family`, `Member`...) et on teste le `from` de chaque règle.
- Phase ② : on **crée** les éléments cibles et on garde le lien source → cible (la **trace**).
- Phase ③ : on **calcule** les liaisons <- ; si une liaison reçoit un élément **source**, ATL met à la place l'élément **cible** correspondant.

Exécution pas à pas sur la famille March

→ Quelle règle pour quel membre ?



Membre	isFemale()	Règle	fullName
Jim (father)	false	Member2Male	Jim March
Cindy (mother)	true	Member2Female	Cindy March
Brandon (sons)	false	Member2Male	Brandon March
Brenda (daughters)	true	Member2Female	Brenda March

Question flash

?

On remplace le filtre de `Member2Male` par `(true)` sans toucher à `Member2Female`. Que se passe-t-il pour **Cindy** ? Et si on supprime complètement la règle `Member2Female` ?

Question flash

? On remplace le filtre de `Member2Male` par `(true)` sans toucher à `Member2Female`. Que se passe-t-il pour **Cindy** ? Et si on supprime complètement la règle `Member2Female` ?

Réponse

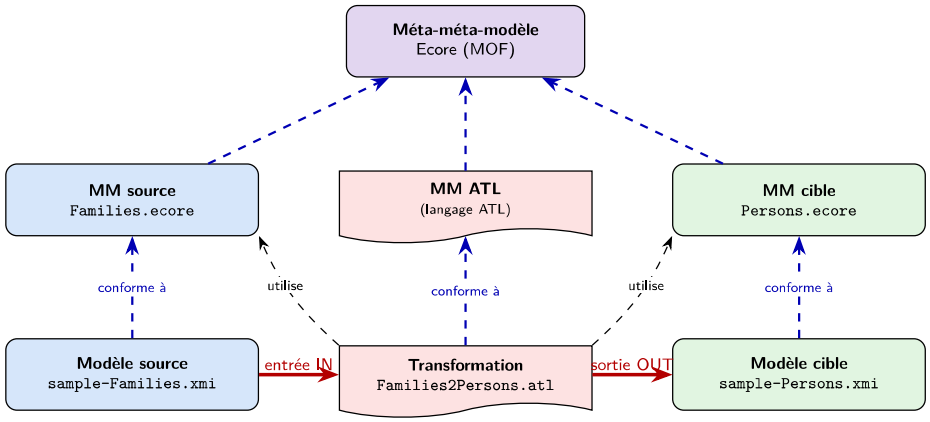
Avec `(true)`, Cindy satisfait **deux** règles : ATL signale une **erreur** à l'exécution (un élément ne peut être pris que par une seule règle). Sans `Member2Female`, les femmes **ne sont pas transformées** : le modèle cible ne contient que les 5 hommes.

Cas complet : Families2Persons

Comment réaliser, de bout en bout, la transformation Families2Persons ?

Les étapes du tutoriel

→ Ce que vous ferez en TP

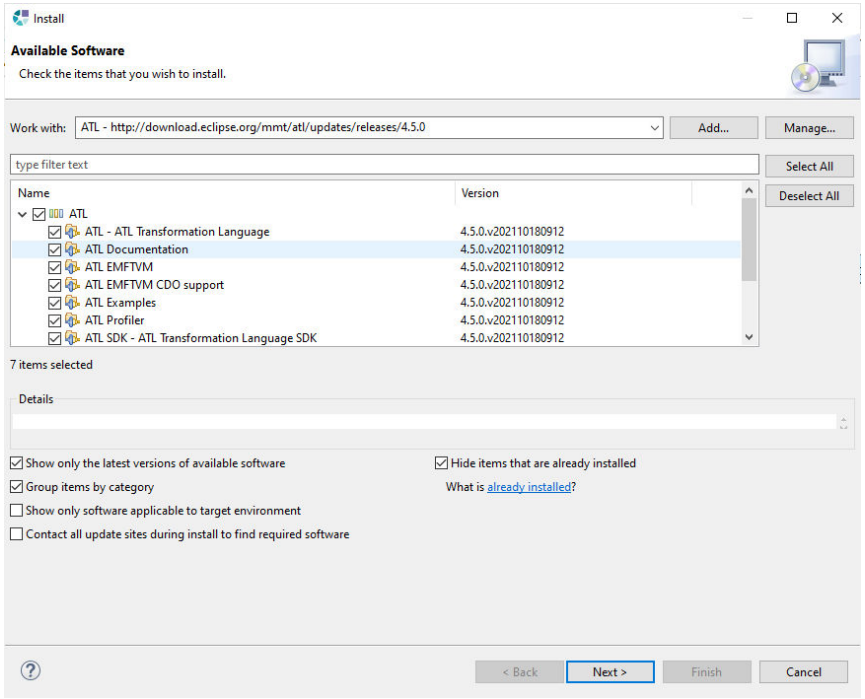


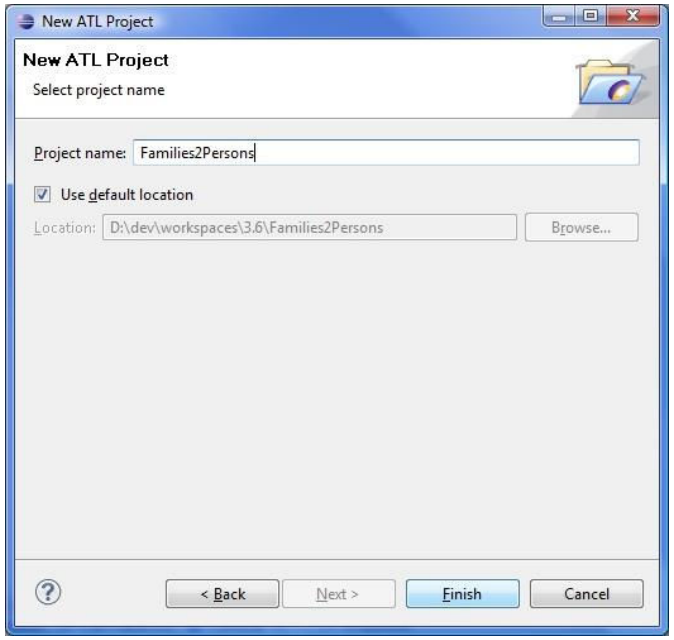
*La transformation est elle-même un **modèle** : tout est conforme à Ecore.*

Étape	Ce que l'on fait	Fichier produit
0	Installer ATL dans Eclipse	–
1	Créer un projet ATL	`Families2Persons/`
2	Créer les deux méta-modèles	`Families.ecore`, `Persons.ecore`
3	Écrire la transformation	`Families2Persons.atl`
4	Écrire un modèle d'exemple	`sample-Families.xmi`
5-6	Configurer et lancer l'exécution	`sample-Persons.xmi`

Étape 0 et 1 : installer ATL, créer le projet

→ Help → Install New Software... puis File → New → Other... → ATL → ATL Project



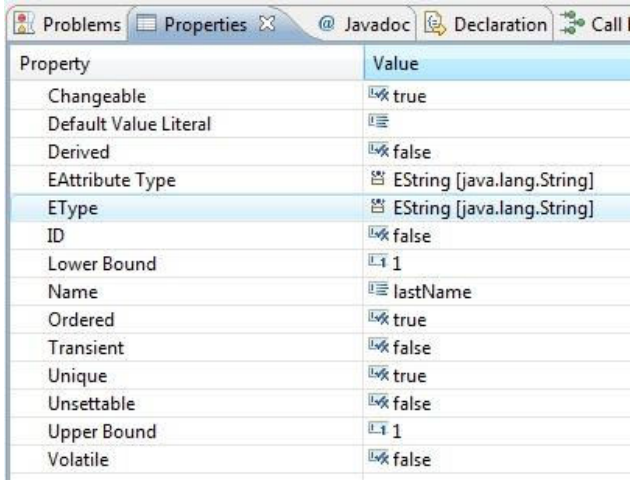


- Choisir le site de mise à jour d'ATL, cocher **ATL SDK**.
- Projet : **ATL Project**, nom `Families2Persons`.

Étape 2 : créer le méta-modèle Families

→ File → New → Other. . . → Eclipse Modeling Framework → Ecore Model

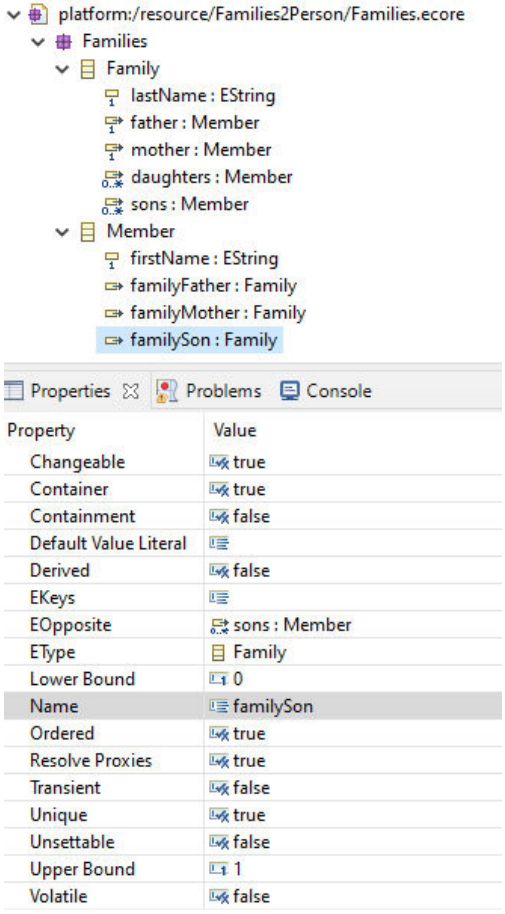




- Nommer le paquetage racine `Families`.
- Clic droit → **New Child** → **EClass** : `Family`, puis `Member`.
- Clic droit sur `Family` → **New Child** → **EAttribute** : `lastName`, **EType** `EString`, bornes **1..1**.
- Dans `Member` : `firstName : EString [1..1]`.
- Les valeurs se saisissent dans la vue **Properties** (Window → Show View → Properties).

Étape 2 : les références et leurs opposées

→ Family → Member (compositions) et Member → Family (opposées)



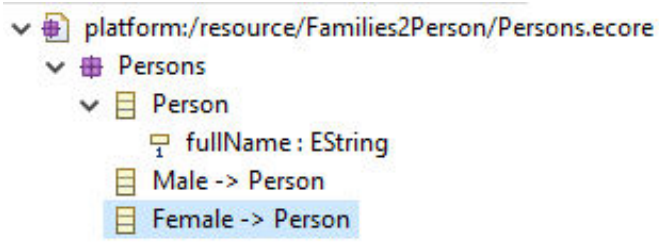
Dans	Référence	Type	Bornes	Containment
Family	`father`, `mother`	Member	1..1	true
Family	`sons`, `daughters`	Member	0..-1 (*)	true
Member	`familyFather` ... `familyDaughter`	Family	0..1	false

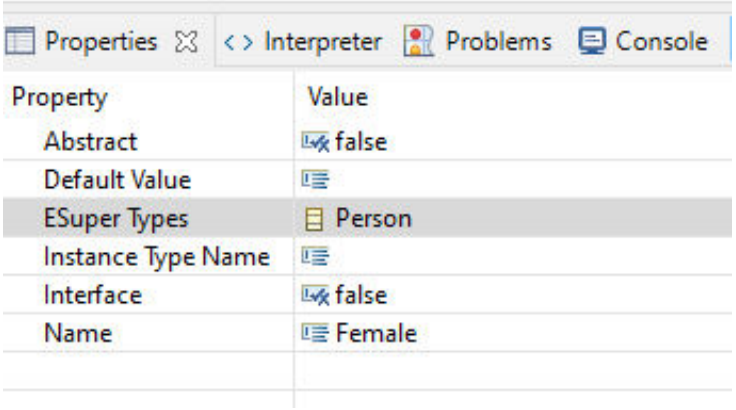
- Renseigner **EOpposite** : `familyFather` ↔ `father`, `familyMother` ↔ `mother`, `familySon` ↔ `sons`, `familyDaughter` ↔ `daughters`.

> **!** Grâce aux opposées, un `Member` **connaît** sa famille et son rôle : c'est ce qu'utilisent les helpers.

Étape 2 : le méta-modèle Persons

→ Plus simple : une classe abstraite et deux sous-classes





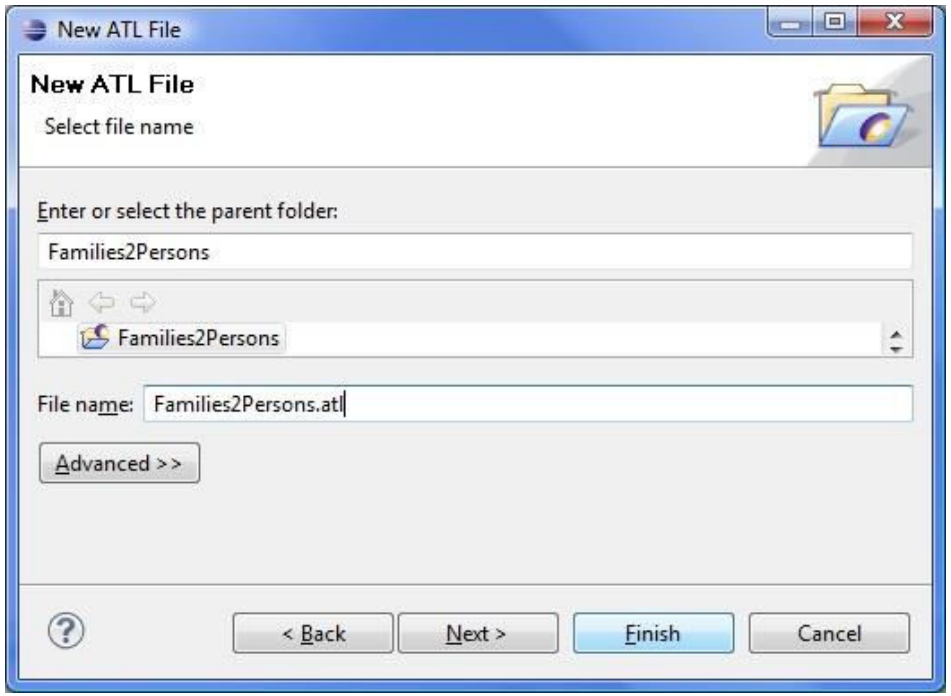
- Paquetage racine `Persons`, classe `Person` avec `fullName : EString [1..1]`.
- Propriété **Abstract** de `Person` à `true`.
- Classes `Male` et `Female` au même niveau ; propriété **ESuper Types** = `Person`.

Erreur classique

Oublier `Abstract = true` n'empêche pas l'exécution, mais oublier **ESuper Types** fait disparaître l'attribut `fullName` de `Male` et `Female` : les liaisons échouent.

Étape 3 : le fichier ATL

→ File → New → Other. . . → ATL → ATL File



- Nom : `Families2Persons.atl`, dans le projet.
- Accepter d'ouvrir la **perspective ATL**.
- Le fichier contient `module Families2Persons;` et une **erreur** : il manque l'en-tête `create ... from ...`.
- Ajouter, dans l'ordre :
  - les deux lignes - `@path`
  - l'en-tête `create OUT : Persons from IN : Families;`
  - les deux helpers
  - les deux règles

Étape 4 : le modèle source

→ File → New → File : sample-Families.xmi (ouvert avec un éditeur de texte)

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<xmi:XMI xmi:version="2.0" xmlns:xmi="http://www.omg.org/XMI" xmlns="Families">
  <Family lastName="March">
    <father firstName="Jim"/>
    <mother firstName="Cindy"/>
    <sons firstName="Brandon"/>
    <daughters firstName="Brenda"/>
  </Family>
  <Family lastName="Sailor">
    <father firstName="Peter"/>
    <mother firstName="Jackie"/>
    <sons firstName="David"/>
    <sons firstName="Dylan"/>
    <daughters firstName="Kelly"/>
  </Family>
</xmi:XMI>
```

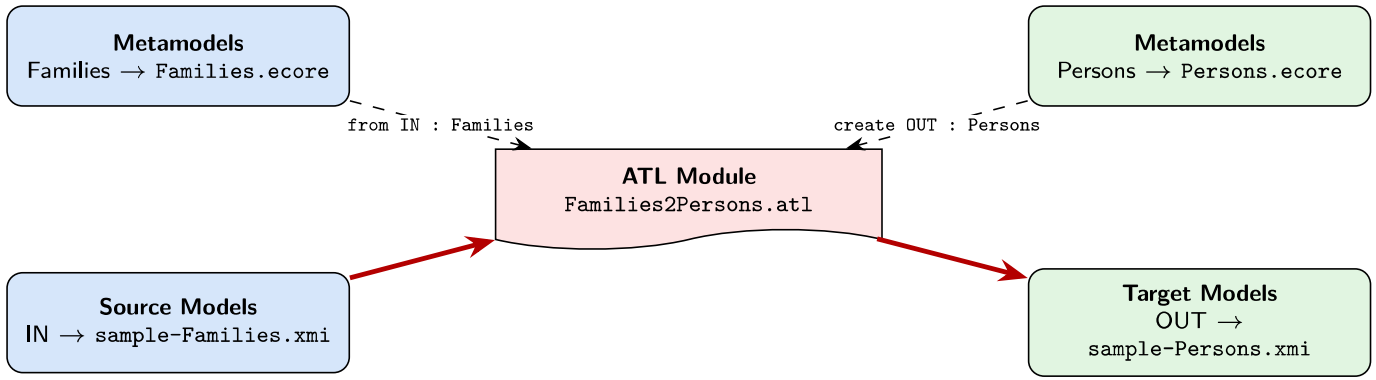
- La racine `xmi:XMI` contient **deux** familles.
- `xmlns="Families"` : le modèle est conforme au méta-modèle **Families** (son `nsURI`).
- Les balises sont les **noms des références** : `father`, `mother`, `sons`, `daughters`.

Corrections du tutoriel

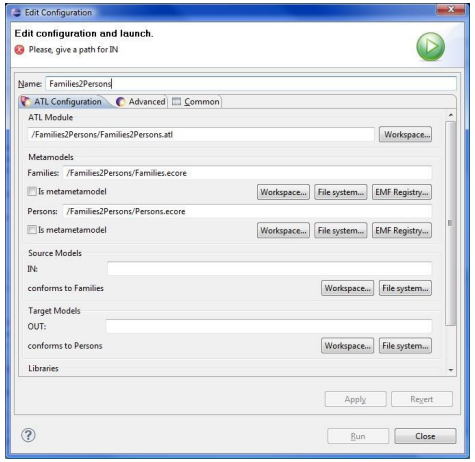
Écrire `<daughters firstName="Kelly"/>` (et non `<filles ...>`) et ne **pas** mettre d'espaces dans les guillemets des espaces de noms : `"http://www.omg.org/XMI"`.

Étape 5 : configurer l'exécution

→ Dans Families2Persons.atl : Run → Run Configurations... → ATL Transformation



*Les noms `IN`, `OUT`, `Families`, `Persons` de l'en-tête doivent **correspondre** à la configuration (Run → Run Configurations).*



- **Source Models** : `IN` → `sample-Families.xmi` (bouton Workspace...).
- **Target Models** : `OUT` → nom du fichier à créer : `sample-Persons.xmi`.
- Onglet **Common** → **Shared file** : enregistre `Families2Persons.launch` pour la prochaine fois.

Étape 6 : exécuter et vérifier

→ Run : le fichier sample-Persons.xmi apparaît dans le projet (F5)

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<xmi:XMI xmi:version="2.0" xmlns:xmi="http://www.omg.org/XMI" xmlns="Persons">
  <Male fullName="Jim March"/>
  <Male fullName="Brandon March"/>
  <Male fullName="Peter Sailor"/>
  <Male fullName="David Sailor"/>
  <Male fullName="Dylan Sailor"/>
  <Female fullName="Cindy March"/>
  <Female fullName="Brenda March"/>
  <Female fullName="Jackie Sailor"/>
  <Female fullName="Kelly Sailor"/>
</xmi:XMI>
```

- **9 membres** en entrée → **9 personnes** en sortie : 5 `Male`, 4 `Female`.
- Les familles ont **disparu** : aucune règle ne les transforme.
- L'ordre des éléments peut varier : il ne compte pas.



Ouvrir le résultat avec **Sample Reflective Ecore Model Editor** : chaque objet doit être un `Male` ou un `Female` avec un `fullName`.

Question flash

?

On ajoute la famille **Benali** : père Ahmed, mère Fatima, fils Yassine, filles Salma et Nour. Combien d'éléments contient le nouveau modèle cible, et lesquels sont créés pour cette famille ?

Question flash

? On ajoute la famille **Benali** : père Ahmed, mère Fatima, fils Yassine, filles Salma et Nour. Combien d'éléments contient le nouveau modèle cible, et lesquels sont créés pour cette famille ?

Réponse

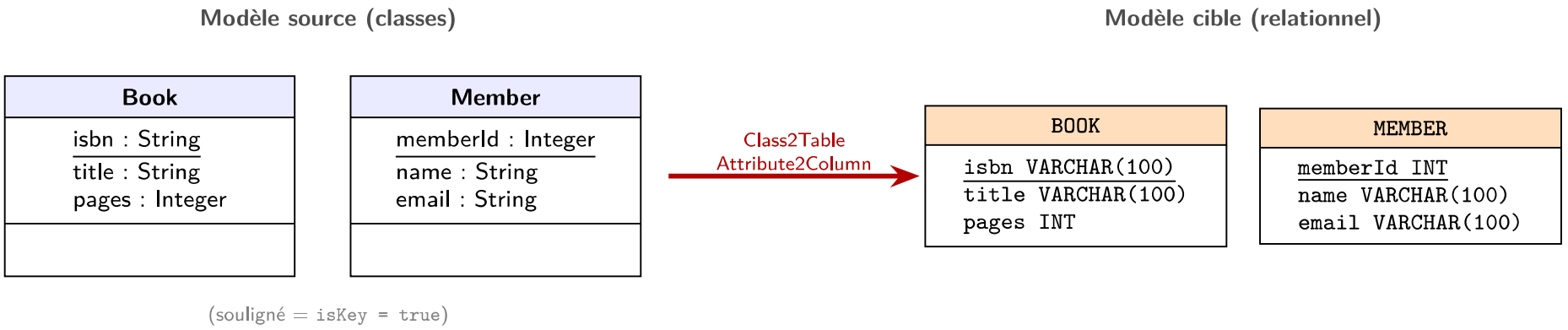
14 personnes (9 + 5). Pour Benali : `Male` « Ahmed Benali », `Male` « Yassine Benali », `Female` « Fatima Benali », `Female` « Salma Benali », `Female` « Nour Benali ». Le module ATL n'est **pas modifié** : il s'applique à tout modèle conforme à Families.

Aller plus loin et synthèse

Comment relier les éléments créés ? Quelles erreurs éviter ? Que retenir ?

Deuxième exemple : des classes aux tables

→ Class2Relational (simplifié) : le PIM → PSM de la Lecture 2



- **Package** → **Schema** ; **Class** (non abstraite) → **Table** (nom en majuscules) ; **Attribute** → **Column** (type traduit en SQL).
- Les attributs marqués `isKey` forment la **clé primaire** de la table.

Class2Relational : le code

→ Un helper sur String et trois règles

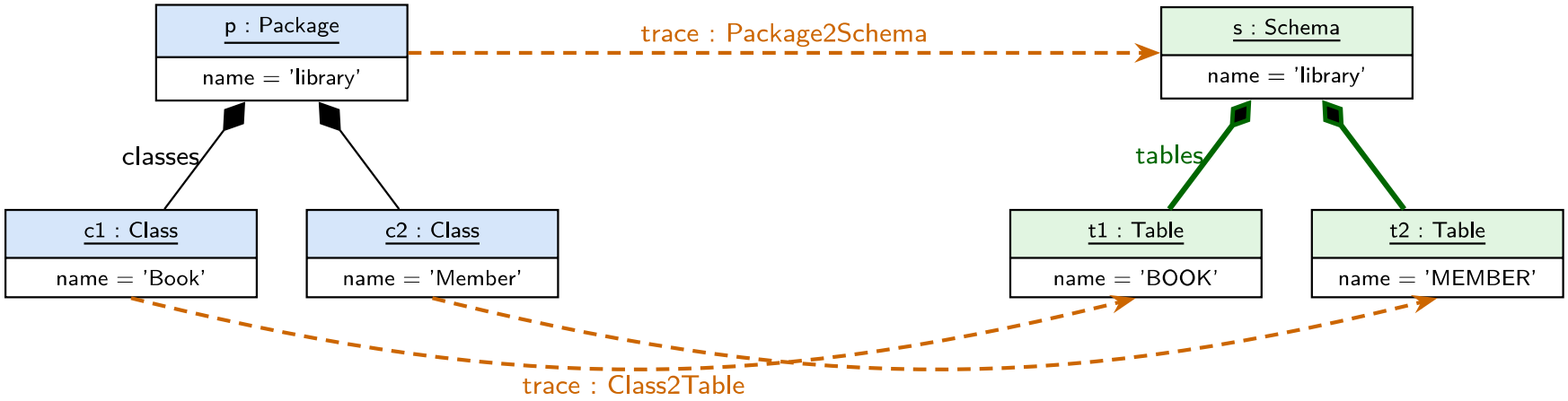

```
module Class2Relational;
create OUT : Relational from IN : SimpleClass;

helper context String def: toSQL() : String =
    if self = 'String' then 'VARCHAR(100)'
    else if self = 'Integer' then 'INT'
    else if self = 'Boolean' then 'BOOLEAN'
    else if self = 'Date' then 'DATE'
    else 'VARCHAR(255)'
    endif endif endif endif;
```

```
rule Package2Schema {
    from p : SimpleClass!Package
    to   s : Relational!Schema (
            name   <- p.name,
            tables <- p.classes )          -- des Class... résolues en Table
}
rule Class2Table {
    from c : SimpleClass!Class (not c.isAbstract)
    to   t : Relational!Table (
            name       <- c.name.toUpper(),
            columns    <- c.attributes,     -- des Attribute... résolus en
        Column
            primaryKey <- c.attributes->select(a | a.isKey) )
}
rule Attribute2Column {
    from a : SimpleClass!Attribute
    to   col : Relational!Column (
            name <- a.name,
            type <- a.type.toSQL() )
}
```

La résolution automatique (la trace)

→ Le point le plus important d'ATL

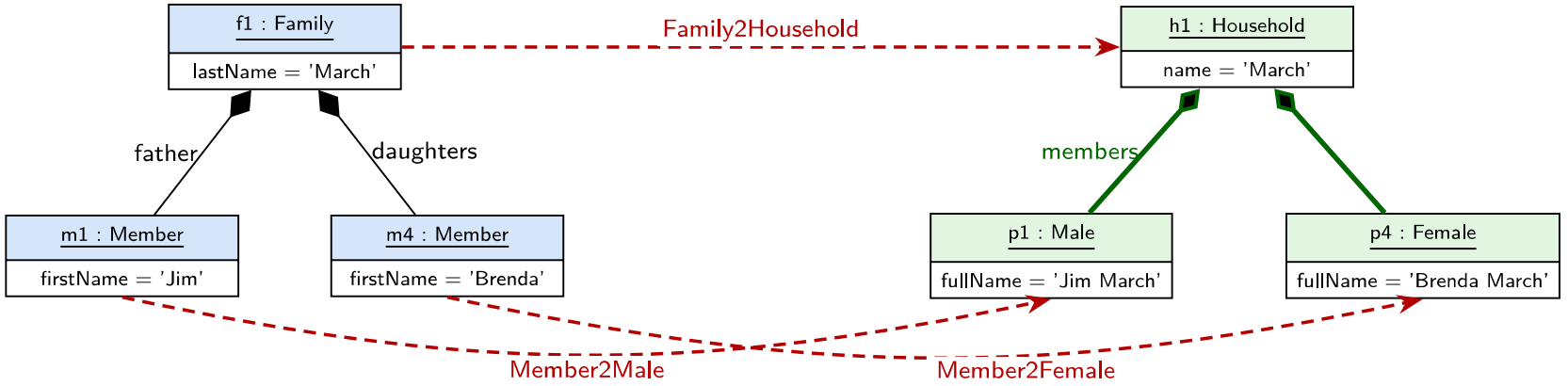



```
tables <- p.classes   - vaut {c1, c2} côté source...
                      - ...ATL affecte {t1, t2} grâce à la trace
```

- `p.classes` contient des **Class** (source) ; mais `tables` attend des **Table** (cible).
- ATL **remplace** automatiquement chaque `Class` par la `Table` créée pour elle par `Class2Table` : on n'écrit **aucun** code pour relier les objets.

Relier les personnes : le foyer

→ Extension du TP : une règle sur Family qui utilise la trace




```
rule Family2Household {
    from
        f : Families!Family
    to
        h : Persons!Household (
            name    <- f.lastName,
            members <- Sequence{f.father, f.mother, f.sons, f.daughters}->flatten()
        )
}
```

- `members` reçoit des `Member` (source) : ATL y met les `Male` / `Female` créés par les deux autres règles.
- `->flatten()` aplatit la séquence de collections en **une seule** séquence.

Les autres sortes de règles

→ Pour information : utiles dans les gros projets

Sorte	Déclenchement	Usage
Matched rule (`rule`)	**Automatique** : pour chaque élément qui satisfait le `from`	Cas général (tout ce cours)
Lazy rule (`lazy rule`)	**Appelée** : `thisModule.R(e)` ; crée un nouvel élément à chaque appel	Créer des copies à la demande
Unique lazy rule	Appelée, mais crée **un seul** élément par élément source	Partager un élément créé
Called rule (`rule R(p : T)`)	Appelée comme une procédure, avec paramètres	Partie impérative (bloc `do { }`)

```
lazy rule Member2Contact {          -- n'est PAS déclenchée automatiquement
    from s : Families!Member
    to   c : Persons!Contact ( label <- s.firstName )
}
-- appel explicite, dans une liaison d'une autre règle :
contacts <- f.sons->collect(m | thisModule.Member2Contact(m))
```

Les erreurs fréquentes

→ Et comment les corriger

Symptôme	Cause	Solution
Erreur sur `module ...`	Nom du module ≠ nom du fichier, ou en-tête `create` absent	`module Families2Persons;` + `create OUT : ... from IN : ...;`
« Several rules match »	Deux filtres vrais pour le même élément	Rendre les filtres **complémentaires**
Modèle cible vide	Mauvais modèle `IN` dans la configuration, ou `xmlns` ≠ `nsURI`	Vérifier Run Configurations et l'espace de noms
« feature ... does not exist »	Nom de propriété mal écrit ou super-type oublié	Comparer avec le méta-modèle (complétion `@path`)
Erreur dans un `if`	`else` ou `endif` oublié	`if ... then ... else ... endif` complet
Chaîne refusée	Guillemets `"..."` au lieu d'apostrophes	Écrire `' '`
Création impossible	Type cible **abstrait** (`Person`)	Créer `Male` ou `Female`

ATL, Acceleo ou programmation ?

→ **Choisir l'outil selon le résultat attendu**

	ATL (M2M)	**Acceleo (M2T)**	**Java + EMF**
Produit	Un **modèle**	Du **texte**	Texte ou modèle
Style	Déclaratif : des **règles**	Template à trous	Impératif : des **boucles**
Parcours du modèle	Fait par le **moteur**	Boucles `[for]`	Écrit à la main
Relier les objets créés	**Automatique** (trace)	–	À programmer (Map)
Typique en MDA	PIM → PSM	PSM → Code	Les deux, cas particuliers

Exercice 1 : un helper isParent

→ Exercice d'application

Question flash

? Écrire un helper **attribut** `isParent : Boolean` sur `Families!Member`, vrai si le membre est le père ou la mère de sa famille. Que vaut-il pour Brandon ?

Exercice 1 : un helper isParent

→ Exercice d'application

Question flash

? Écrire un helper **attribut** `isParent : Boolean` sur `Families!Member`, vrai si le membre est le père ou la mère de sa famille. Que vaut-il pour Brandon ?

Réponse

Pour Brandon (fils) : `false`.

```
helper context Families!Member def: isParent : Boolean =
    not self.familyFather.oclIsUndefined()
    or not self.familyMother.oclIsUndefined();
```

Exercice 2 : compter les enfants

→ Exercice d'application

Question flash

?

Écrire un helper `nbChildren : Integer` sur `Families!Family`. Que vaut-il pour la famille **Sailor** ?

Exercice 2 : compter les enfants

→ Exercice d'application

Question flash

? Écrire un helper `nbChildren : Integer` sur `Families!Family`. Que vaut-il pour la famille **Sailor** ?

Réponse

3 pour Sailor (David, Dylan, Kelly).

```
helper context Families!Family def: nbChildren : Integer =
    self.sons->size() + self.daughters->size();
```

Exercice 3 : changer le format du nom

→ Exercice d'application

Question flash

? Modifier `Member2Female` pour obtenir des noms de la forme « **MARCH, Cindy** ». Que devient « Kelly Sailor » ?

Exercice 3 : changer le format du nom

→ Exercice d'application

Question flash

? Modifier `Member2Female` pour obtenir des noms de la forme **« MARCH, Cindy »**. Que devient « Kelly Sailor » ?

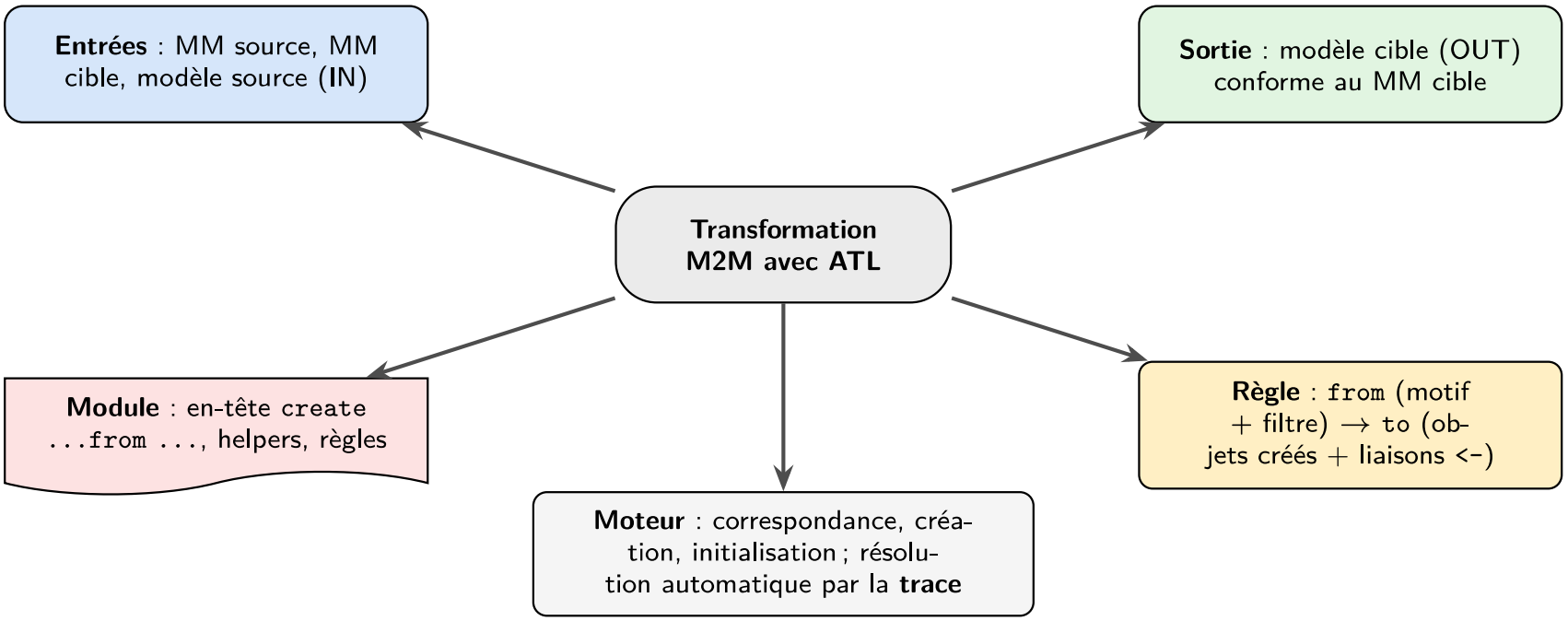
Réponse

« SAILOR, Kelly ». Seule la **liaison** change :

```
rule Member2Female {
    from s : Families!Member (s.isFemale())
    to   t : Persons!Female (
             fullName <- s.familyName.toUpper() + ', ' + s.firstName )
}
```

Synthèse

→ La transformation M2M avec ATL en une image



Aide-mémoire ATL

→ La syntaxe à connaître

```
module M;  create OUT : MMcible from IN : MMsource;

helper context MMsource!Classe def: nom : Type = <expression OCL>;
helper context MMsource!Classe def: op(p : Type) : Type = <expression OCL>;

rule R {
    from s : MMsource!ClasseSource (<filtre booléen>)
    to   t : MMcible!ClasseCible ( attr <- <expression sur s>,
                                   ref  <- <élément(s) source : résolu(s) par la trace> )
}
```

Mot-clé	Rôle
`module`, `create ... from ...`	Nom et modèles IN / OUT
`helper context ... def:`	Fonction OCL (`self` = l'objet)
`rule ... { from ... to ... }`	Correspondance source → cible
`<-`	Liaison : donner une valeur à une propriété cible
`MM!Classe`	Une classe du méta-modèle MM
`thisModule`	Le module (helpers sans contexte, règles appelées)

Pour la suite et références

→ TD, TP et documentation

- **TD** : lire, compléter et écrire des modules ATL (Families2Persons, Class2Relational).
- **TP** : le tutoriel complet dans Eclipse, puis trois extensions (nouvelle famille, attribut `role`, foyers).
- Références :
  - Projet ATL (Eclipse) : `eclipse.dev/atl` – téléchargements, documentation, exemples (dont Families2Persons).
  - ATL User Guide (wiki Eclipse) et « ATL User Manual » (dossier documentation IDM).
  - F. Jouault, F. Allilaire, J. Bézivin, I. Kurtev, « ATL : A model transformation tool », Science of Computer Programming, 2008.
  - Tutoriel « TP3 : Transformation par modélisation (ATL) », Pr. A. Berrahou.

> **!** Une transformation ATL = **des règles sur les méta-modèles** ; le moteur parcourt le modèle, crée les éléments et les **relie** grâce à la trace.